

Lab Assignment – 7.5

Hall Ticket No.: 2303A510E6

Name – Anushka Boora

Batch – 29

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):
```

```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.

Code:

Bug: Mutable default argument

```
def add_item(item, items=None):
```

```
    if items is None:
```

```
        items = []
```

```
    items.append(item)
```

```
    return items
```

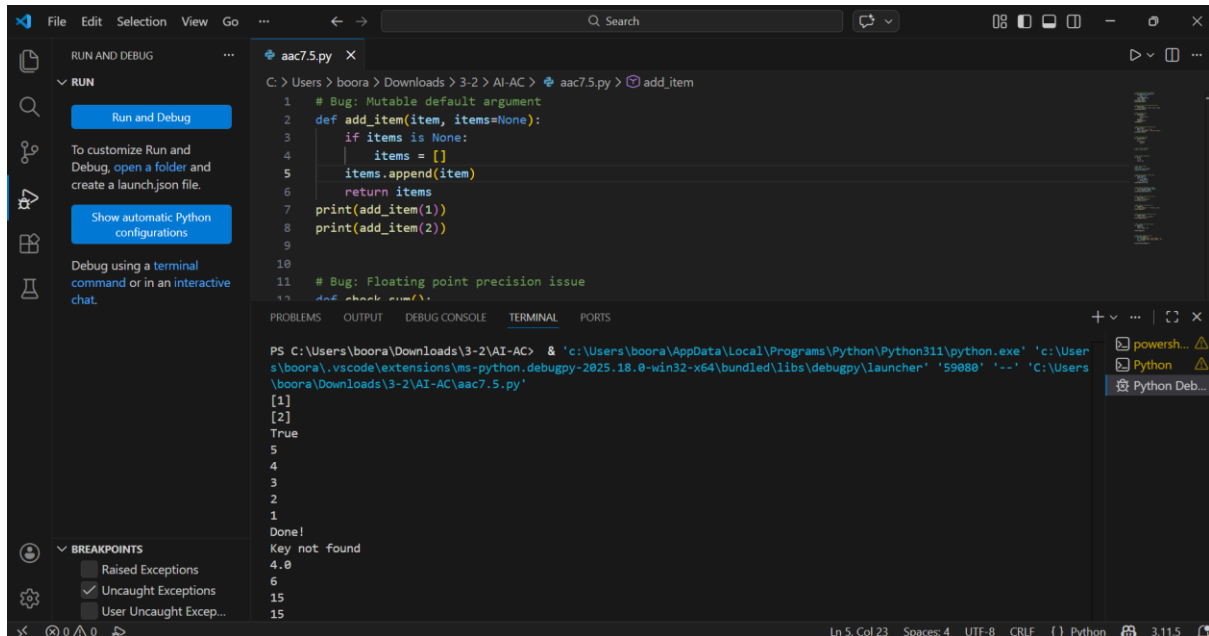
```
print(add_item(1))
```

```
print(add_item(2))
```

Output:

[1]

[2]



Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance.

Bug: Floating point precision issue

```
def check_sum():
```

```
    return (0.1 + 0.2) == 0.3
```

```
print(check_sum())
```

Expected Output: Corrected function

Code:

Bug: Floating point precision issue

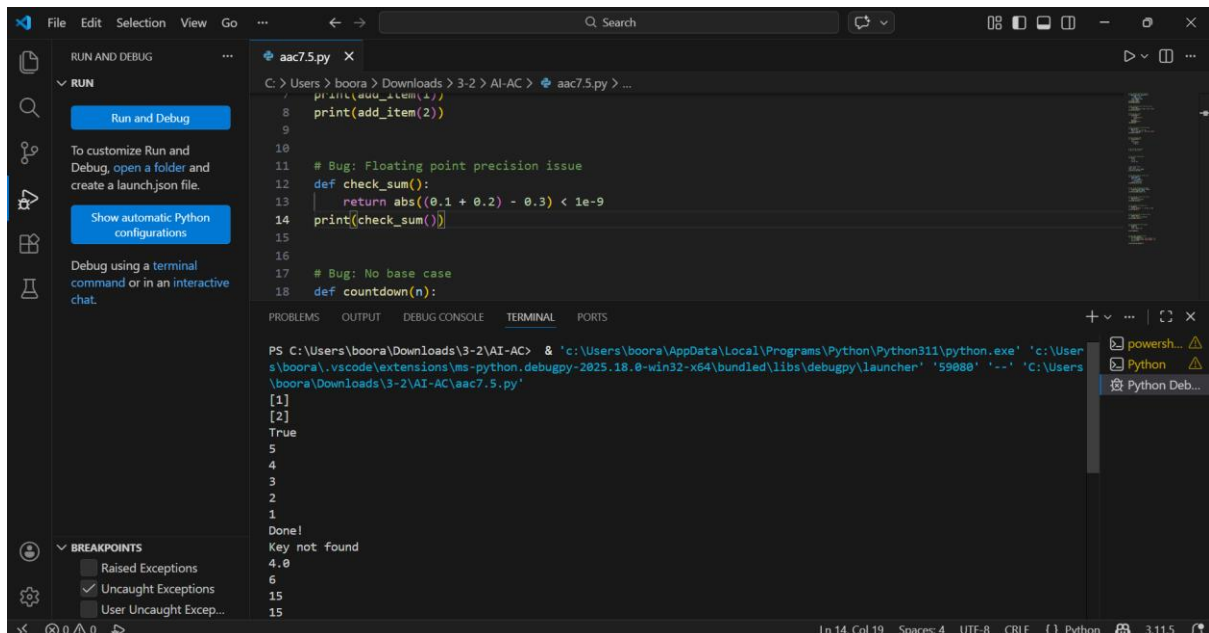
```
def check_sum():
```

```
    return abs((0.1 + 0.2) - 0.3) < 1e-9
```

```
print(check_sum())
```

Output:

True



```
File Edit Selection View Go ... Search
C:\Users\boora> Downloads\3-2\AI-AC> aac7.5.py > ...
8 print(add_item(1))
9
10
11 # Bug: Floating point precision issue
12 def check_sum():
13     return abs((0.1 + 0.2) - 0.3) < 1e-9
14 print(check_sum())
15
16
17 # Bug: No base case
18 def countdown(n):
19     print(n)
20     return countdown(n-1)
21
22 countdown(5)
23
24 Done!
25 Key not found
26 4.0
27 6
28 15
29 15
```

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case

```
def countdown(n):
```

```
    print(n)
```

```
    return countdown(n-1)
```

```
countdown(5)
```

Expected Output : Correct recursion with stopping condition.

Code:

Bug: No base case

```
def countdown(n):
```

```

if n <= 0:
    print("Done!")
    return
print(n)
countdown(n-1)
countdown(5)

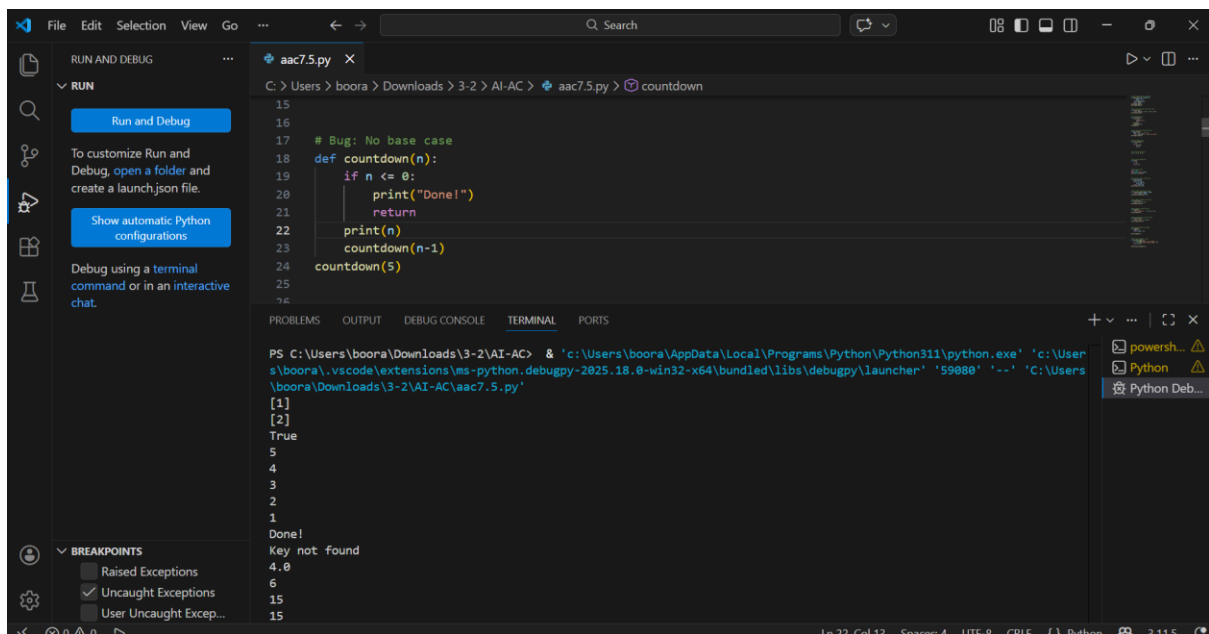
```

Output:

```

5
4
3
2
1
Done!

```



Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():

    data = {"a": 1, "b": 2}

    return data["c"]

print(get_value())
```

Expected Output: Corrected with .get() or error handling.

Code:

Bug: Accessing non-existing key

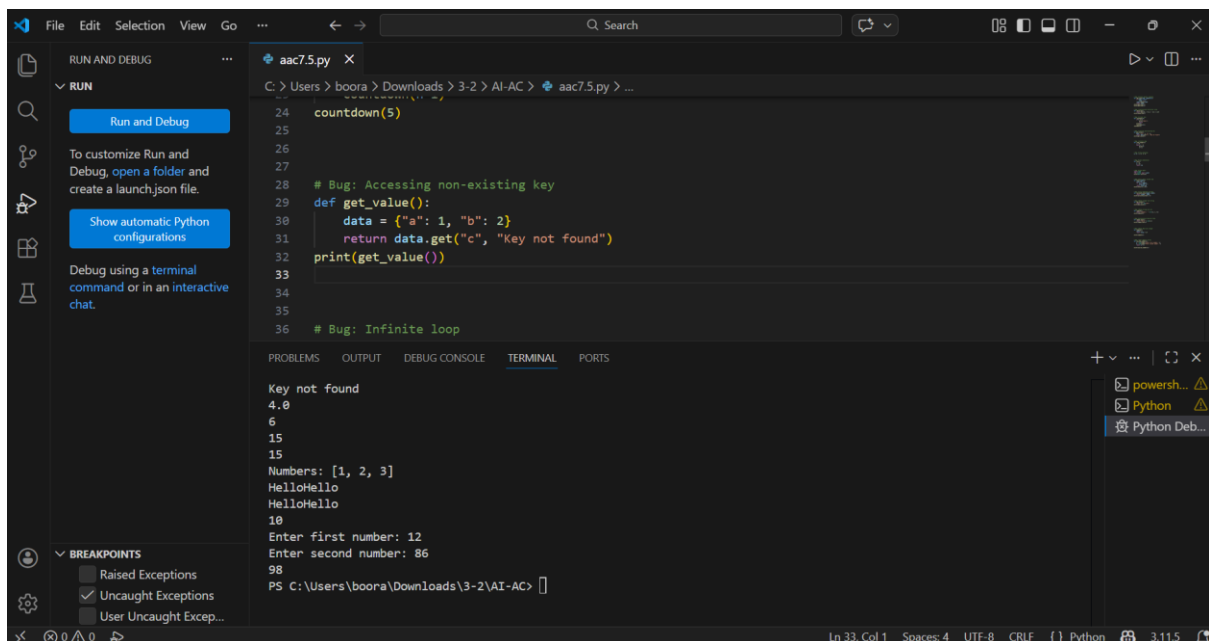
```
def get_value():

    data = {"a": 1, "b": 2}

    return data.get("c", "Key not found")

print(get_value())
```

Output:



Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():
```

```
    i = 0
```

```
    while i < 5:
```

```
        print(i)
```

Expected Output: Corrected loop increments i.

Code:

Bug: Infinite loop

```
def loop_example():
```

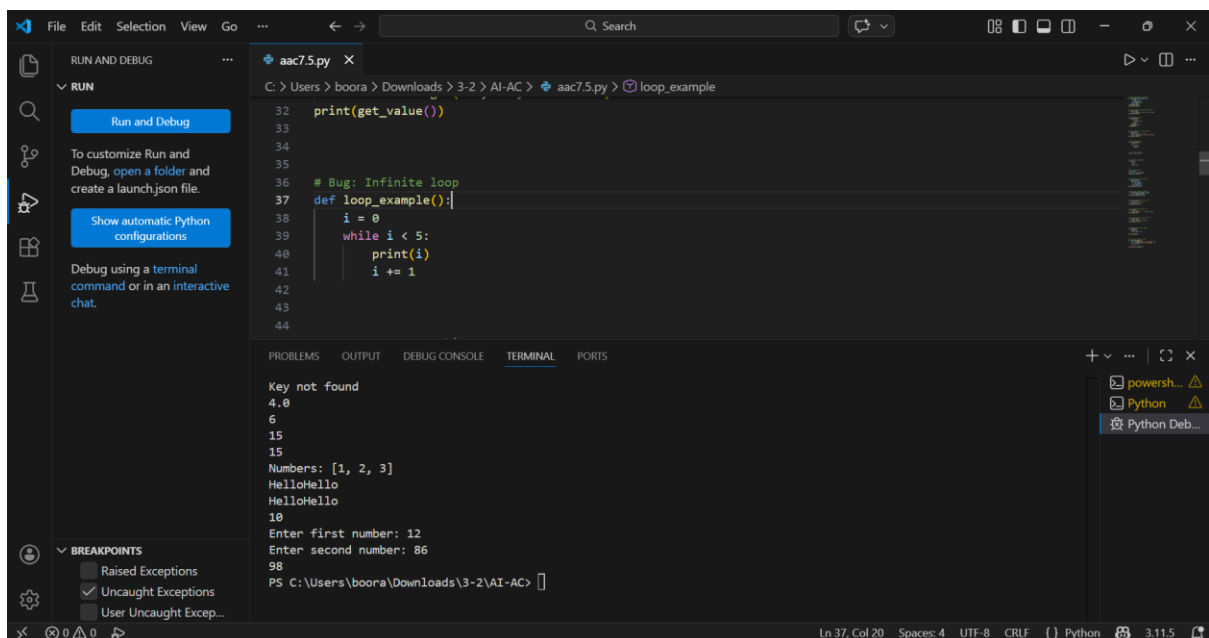
```
    i = 0
```

```
    while i < 5:
```

```
        print(i)
```

```
        i += 1
```

Output:



Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

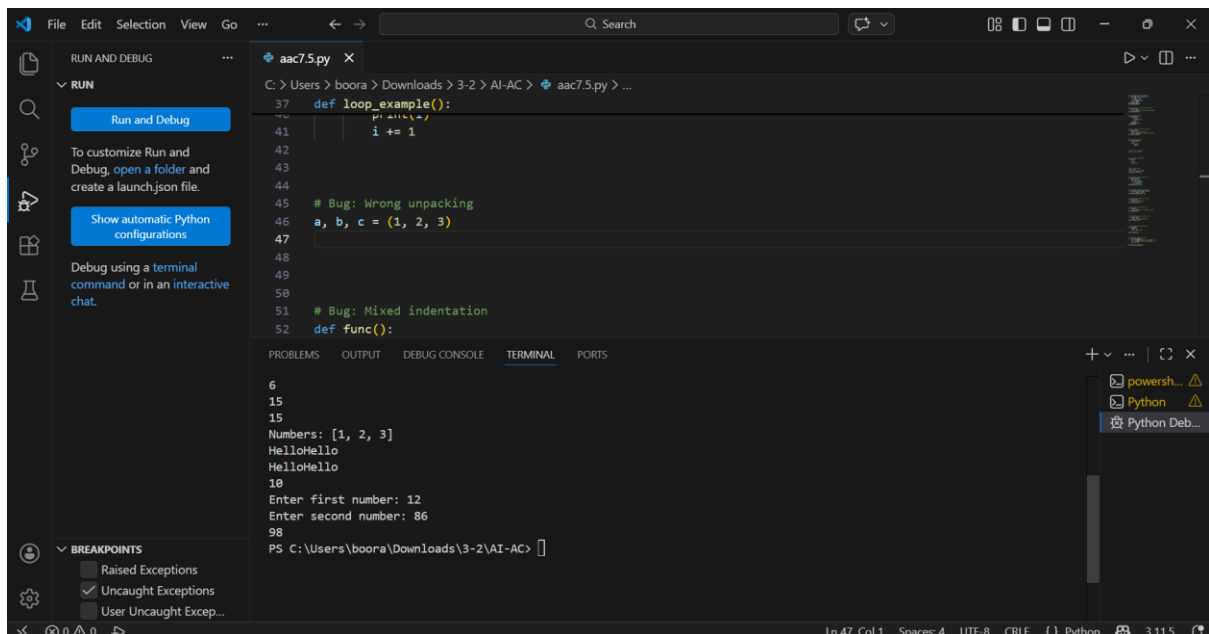
Expected Output: Correct unpacking or using _ for extra values.

Code:

Bug: Wrong unpacking

a, b, c = (1, 2, 3)

Output:



Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

def func():

 x = 5

 y = 10

 return x+y

Expected Output : Consistent indentation applied.

Code:

Bug: Mixed indentation

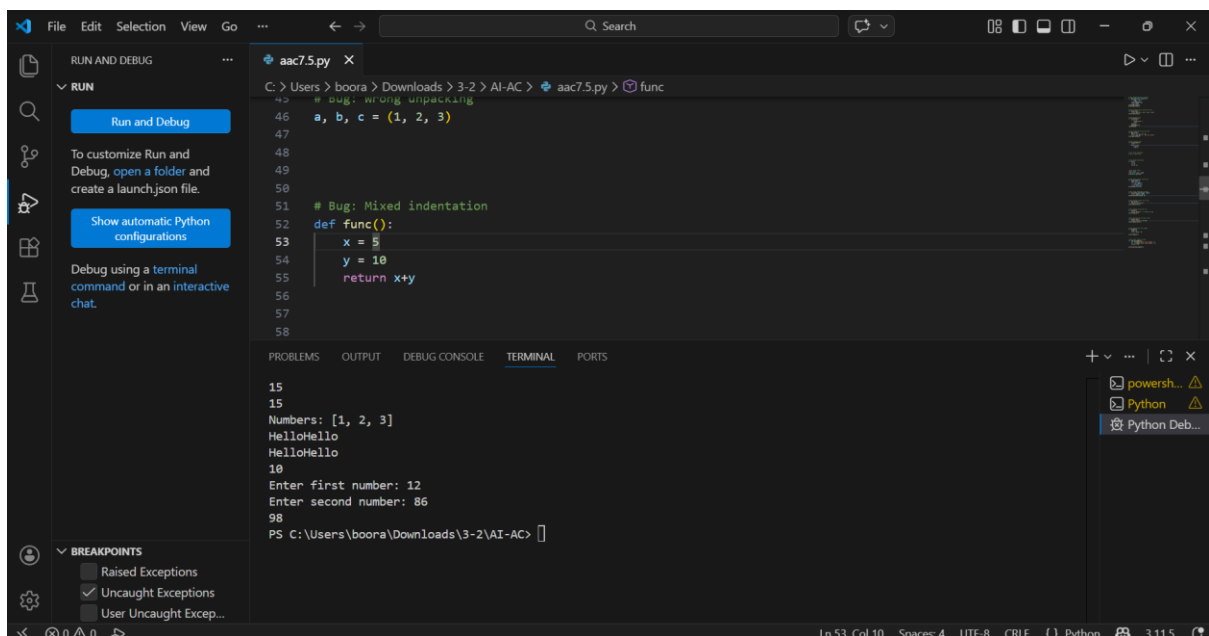
```
def func():
```

```
    x = 5
```

```
    y = 10
```

```
    return x+y
```

Output:



Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
```

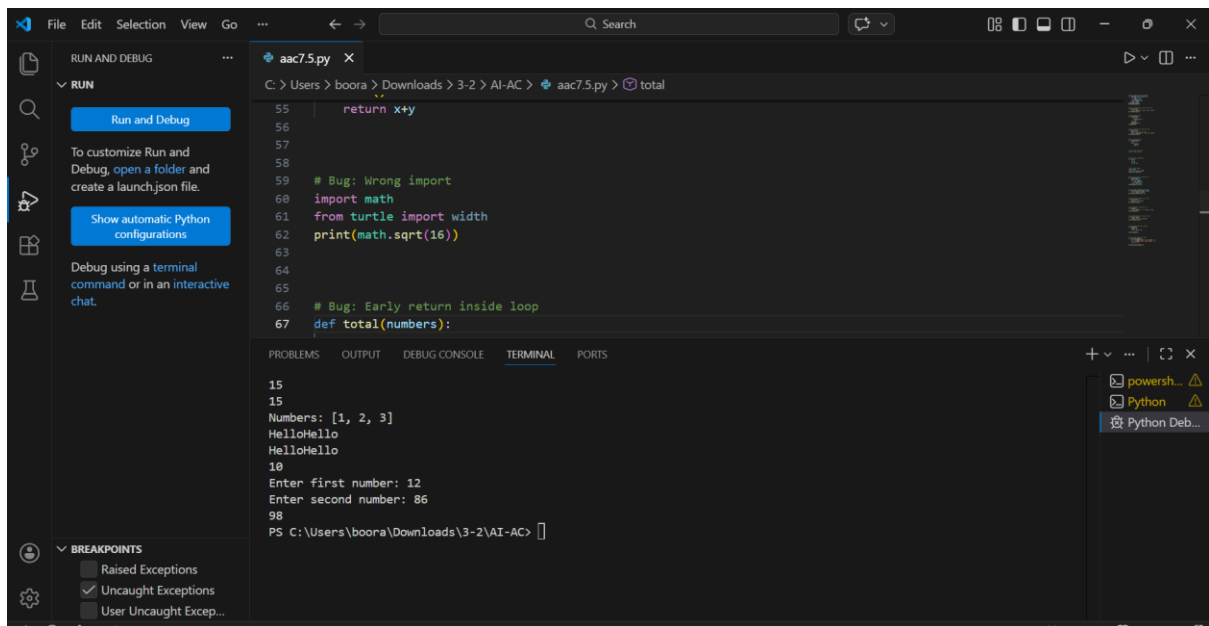
```
print(maths.sqrt(16))
```

Expected Output: Corrected to import math

Code:

```
# Bug: Wrong import  
  
import math  
  
from turtle import width  
  
print(math.sqrt(16))
```

Output:



Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

```
def total(numbers):
```

```
    for n in numbers:
```

```
        return n
```

```
print(total([1,2,3]))
```

Expected Output: Corrected code accumulates sum and returns after loop.

Code:

Bug: Early return inside loop

```
def total(numbers):
```

```
    total_sum = 0
```

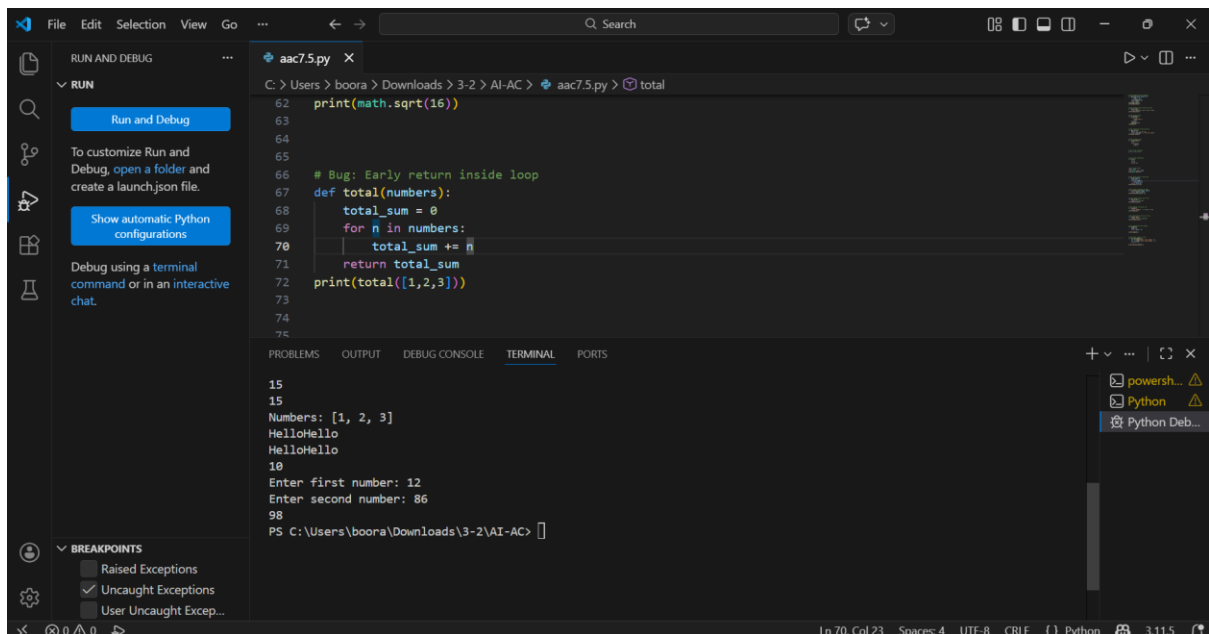
```
    for n in numbers:
```

```
        total_sum += n
```

```
    return total_sum
```

```
print(total([1,2,3]))
```

Output:



Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():
```

```
    return length * width
```

```
print(calculate_area())
```

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.

Code:

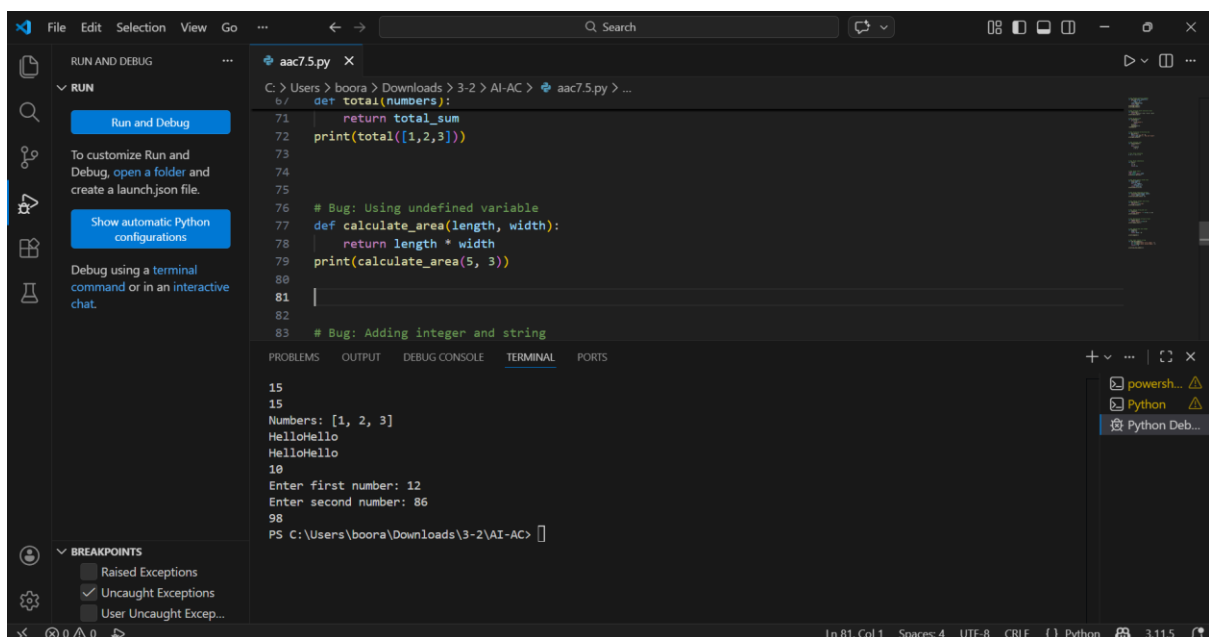
Bug: Using undefined variable

```
def calculate_area(length, width):
```

```
    return length * width
```

```
print(calculate_area(5, 3))
```

Output:



Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```
def add_values():  
    return 5 + "10"  
  
print(add_values())
```

Requirements:

- Run the code to observe the error.
- AI should explain why int + str is invalid.
- Fix the code by type conversion (e.g., int("10") or str(5)).
- Verify with 3 assert cases.

Expected Output #6:

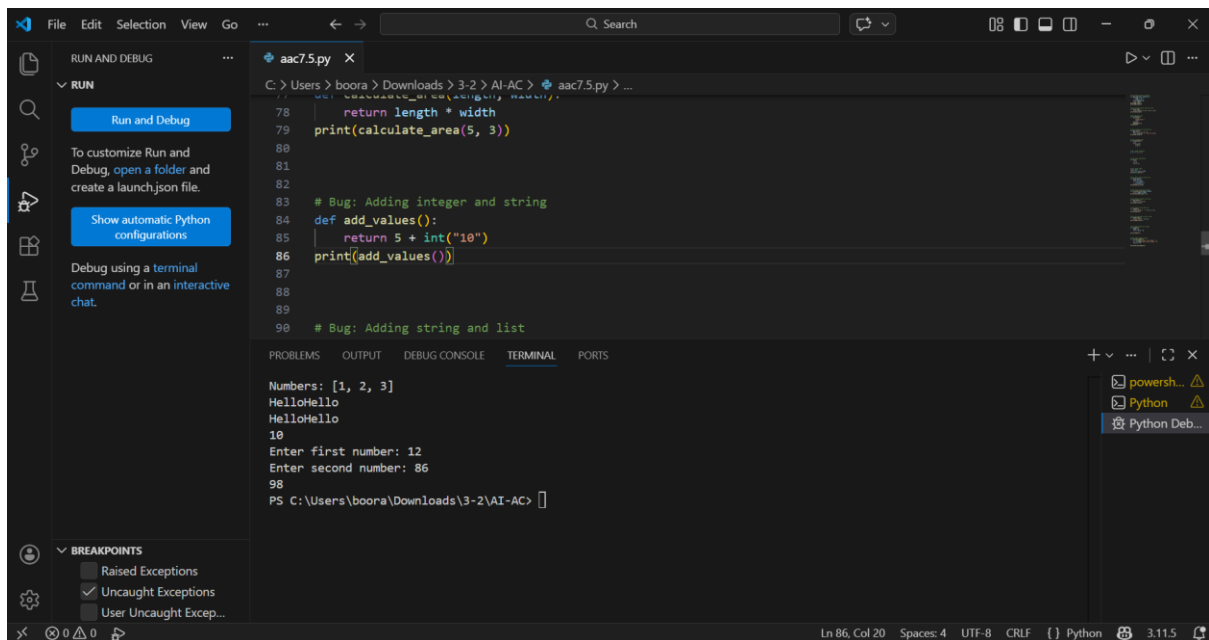
- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.

Code:

```
# Bug: Adding integer and string  
  
def add_values():  
    return 5 + int("10")  
  
print(add_values())
```

Output:



Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

def combine():

return "Numbers: " + [1, 2, 3]

print(combine())

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

Code:

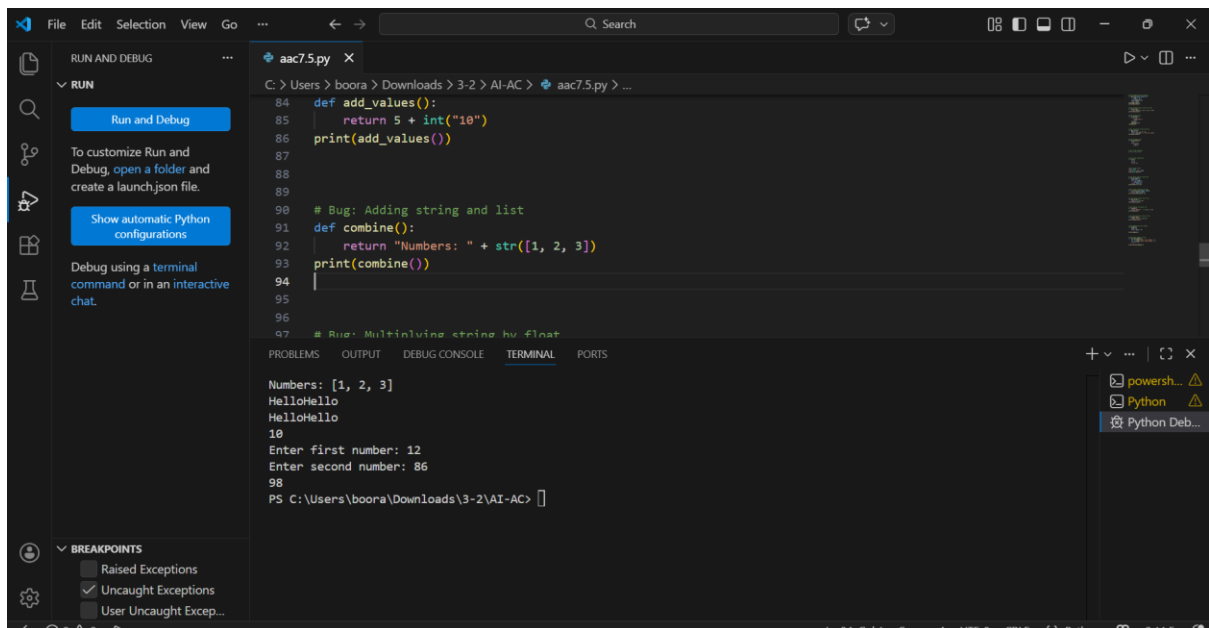
Bug: Adding string and list

def combine():

 return "Numbers: " + str([1, 2, 3])

print(combine())

Output:



Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

Bug: Multiplying string by float

def repeat_text():

 return "Hello" * 2.5

print(repeat_text())

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.

- Add 3 assert test cases.

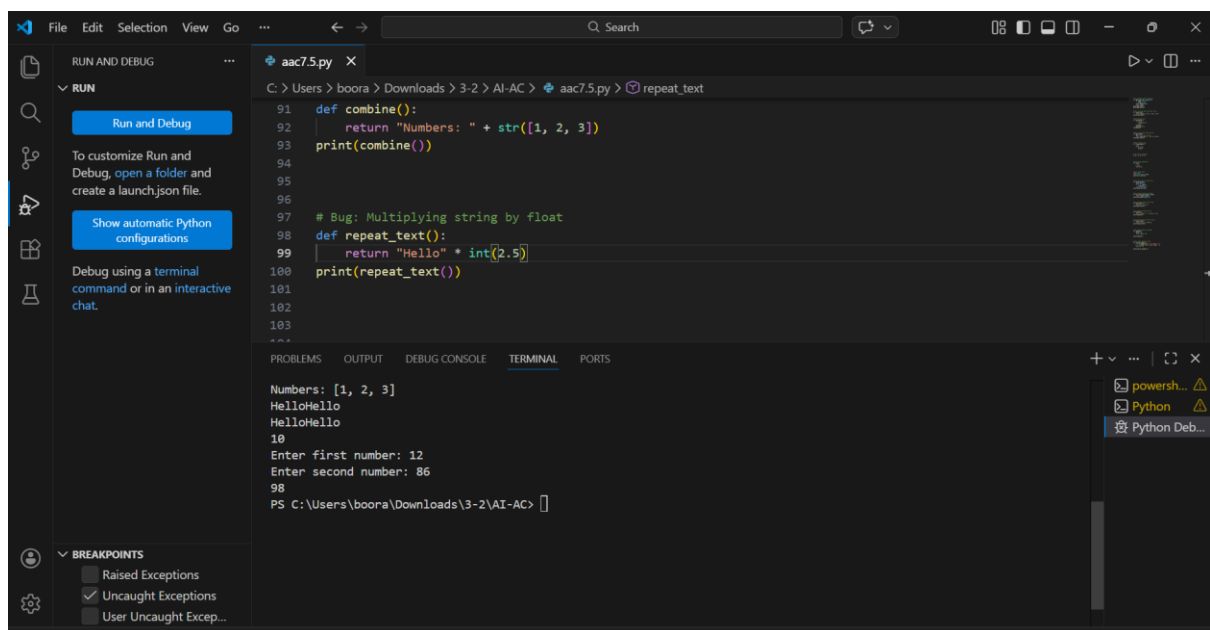
Code:

Bug: Multiplying string by float

```
def repeat_text():
    return "Hello" * int(2.5)

print(repeat_text())
```

Output:



Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

```
def compute():
    value = None

    return value + 10
```

```
print(compute())
```

Requirements:

- Run and identify the error.
- Explain why `NoneType` cannot be added.
- Fix by assigning a default value.
- Validate using asserts.

Code:

Bug: Adding None and integer

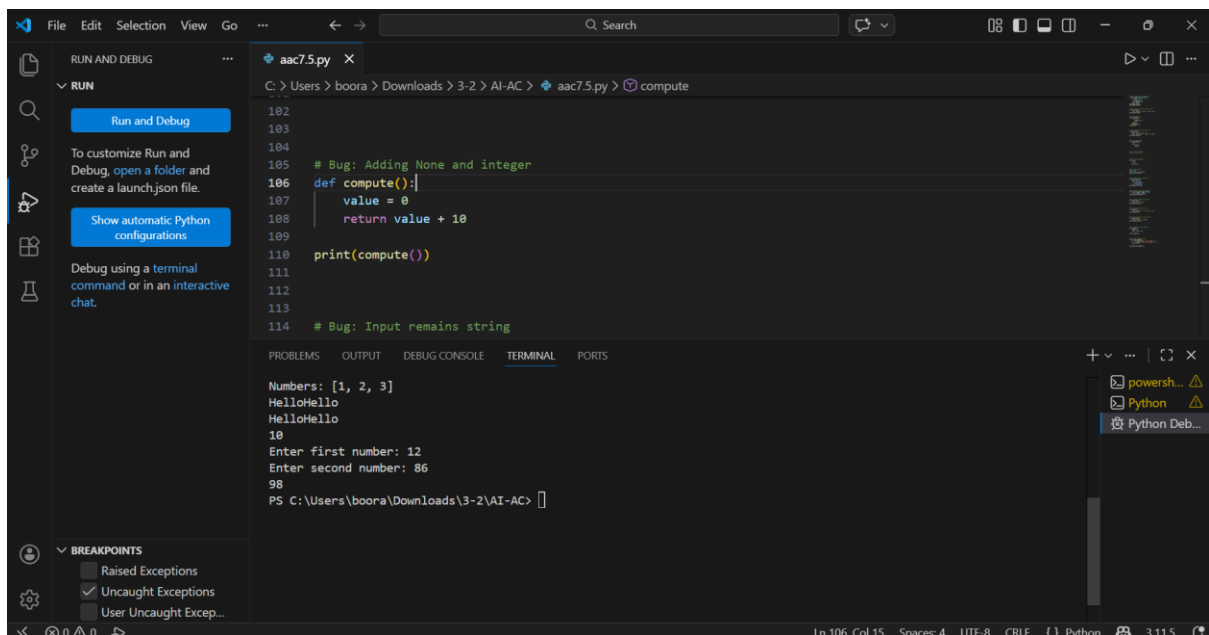
```
def compute():
```

```
    value = 0
```

```
    return value + 10
```

```
print(compute())
```

Output:



Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string

```
def sum_two_numbers():  
    a = input("Enter first number: ")  
    b = input("Enter second number: ")  
    return a + b
```

```
print(sum_two_numbers())
```

Requirements:

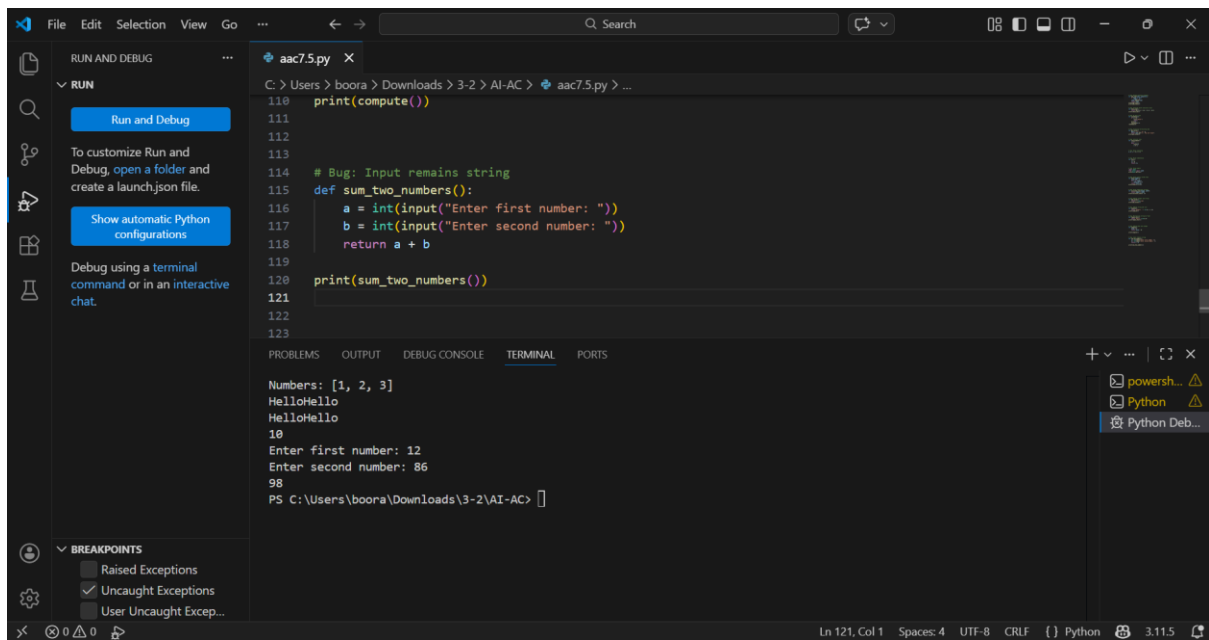
- Explain why input is always string.
- Fix using int() conversion.
- Verify with assert test cases.

Code:

Bug: Input remains string

```
def sum_two_numbers():  
    a = int(input("Enter first number: "))  
    b = int(input("Enter second number: "))  
    return a + b  
  
print(sum_two_numbers())
```

Output:



The screenshot shows a Python IDE with a file named `aac7.5.py` open. The code in the editor is as follows:

```
110 print(compute())
111
112
113
114 # Bug: Input remains string
115 def sum_two_numbers():
116     a = int(input("Enter first number: "))
117     b = int(input("Enter second number: "))
118     return a + b
119
120 print(sum_two_numbers())
121
122
123
```

The left sidebar shows the **RUN AND DEBUG** panel with a **RUN** section containing a **Run and Debug** button and a **DEBUG** section with instructions on how to debug. Below these are **BREAKPOINTS** for **Raised Exceptions**, **Uncaught Exceptions**, and **User Uncaught Exceptions**.

The bottom panel shows the **TERMINAL** output:

```
Numbers: [1, 2, 3]
HelloHello
HelloHello
10
Enter first number: 12
Enter second number: 86
98
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

The status bar at the bottom indicates the current line and column (Ln 121, Col 1), the file encoding (UTF-8), the line ending (CRLF), the file type (Python), and the Python version (3.11.5).